[Bug Report] RSD Processor Does Not Throw Exception When Floating-Point Unit is Disabled

Metainformation

JVN-ID: PENDING

Contact: Feel free to reach out for more details to jv.rajendran@tamu.edu

Discoverers: Chen Chen, Jayden Koh, Stephen Muttathil, Jeyavijayan Rajendran, Texas A&M University; Ahmad-Reza Sadeghi, Technische Universitat Darmstadt;

References:
https://github.com/rsd-devel/rsd/issues/102

Proposed CVSS:
CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:L/SC:N/SI:N/SA:N

CVSS v4.0 Score: **9.3 / Critical**

Macro vector: 000200
Exploitability: High
Complexity: High
Vulnerable system: High
Subsequent system: Low
Exploitation: High
Security requirements: High

Relevant CWEs:
CWE-440: Expected Behavior Violation
CWE-1281: Sequence of Processor Instructions Leads to Unexpected Behavior

Vulnerability Information

Vulnerability type: This vulnerability has implications for incorrect privilege-level transitions and exception handling on the RSD processor.

Vendors of the product: RSD is maintained by Ryota Shioya, The University of Tokyo; Koji Inoue, Kyushu University, and the National Institute of Informatics (Japan), Masahiro Goshima.

Product:
RISC-V Out-Of-Order Superscalar Processor (RSD)
https://github.com/rsd-devel/rsd
commit: 7b65f6b

Has the vendor confirmed or acknowledged the vulnerability? No

Attack Type: Remote

Impact: Escalation of Privileges, Code Execution, Information Disclosure

Affected Components: Software programs utilizing floating-point registers on the RSD processor.

Attack Vectors: Any software program that contains write or read operations to floating-point registers.

Description of the vulnerability: Executing Floating Point Register operations on RSD does not throw an exception when the FS flag in the mstatus register is “Off” (00)

Additional Information:

This vulnerability occurs in the handling of floating-point registers by the RISC-V Out-Of-Order Superscalar Processor (RSD), which allows arbitrary processes to perform illegal operations on any floating-point registers. The GitHub issue describing the vulnerability can be found here.

Technical Information

This vulnerability enables unprivileged users to gain unlimited access to floating-point registers with arbitrary read and write permissions, even when floating-point operations are explicitly disabled.

According to the RISC-V manual:

> "When an extension’s status is set to Off, any instruction that attempts to read or write the corresponding state will cause an illegal-instruction exception." - RISC-V Manual

RSD GitHub Commit: `7b65f6b`

Background

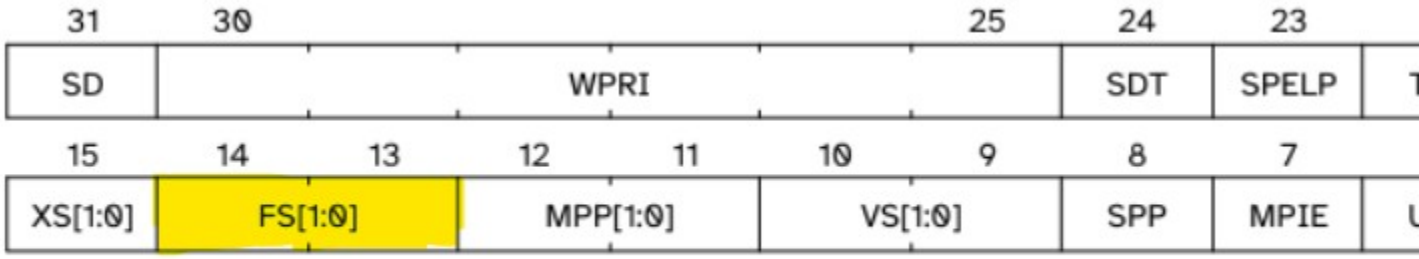


Figure 7. Machine-mode status (mstatu

The specifications for the mstatus register show that the FS flag is in the `mstatus[14:13]`.

Table 11. Encoding of FS[1:0], VS[1:0], and XS[1:0] status fields

Status	FS and VS Meaning	XS Meaning
0	Off	All off
1	Initial	None dirty or clean, some on
2	Clean	None dirty, some clean
3	Dirty	Some dirty

The off status of the FS flag is `00`; floating-point registers should not be accessible when it is off.

In an example program (provided below as exploit_folder.zip), an attempt is made to access a floating-point register while the FS flag is “Off”.

```
1b18:       f00500d3                fmv.w.x ft1,a0
1b1c:       00000013                nop
1b20:       00000013                nop
1b24:       00000013                nop
1b28:       300023f3                csrr    t2,mstatus
```

Figure 1: *Disassembly of the exploit program with relevant instructions.*

The instruction at 0x1b18 performs an operation (read or write access) with a floating-point register (ft0).

Expected Behavior

If the floating point instruction executes when the FS flag in the mstatus register is “Off” (00), this instruction should throw an illegal exception because floating point operations have been disabled.

Actual Behavior

Below is the waveform showing that the processor does not trigger an exception at that instruction and proceeds to execute additional instructions, even though a floating-point register is being accessed. Refer back to the disassembly (**Figure 1**) to verify that a floating-point register is being accessed.

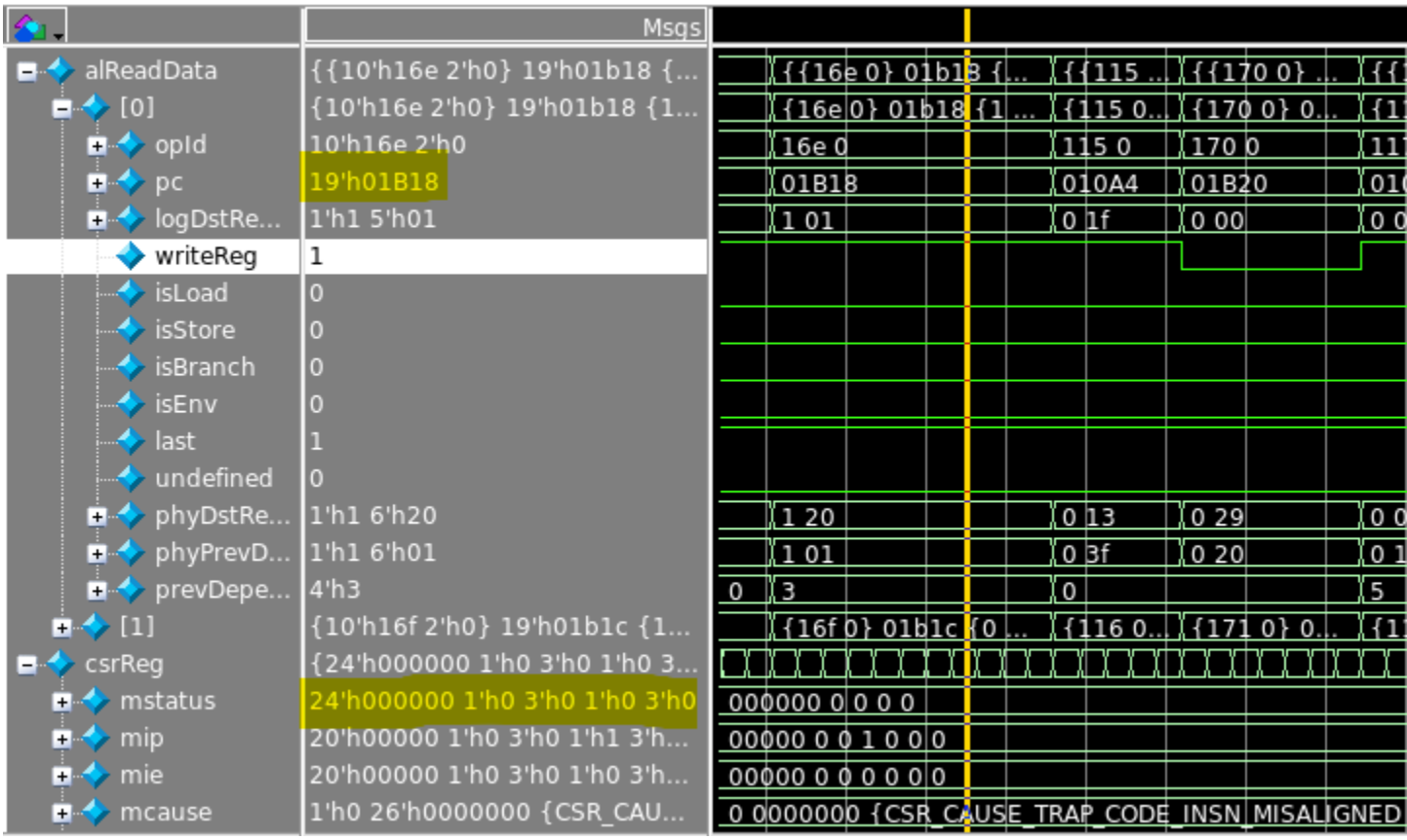


Figure 2: *Waveform of the RSD processor executing illegal instructions without checking the FS flag.*

Steps to Replicate

1. Compiled C code to hex executable. The code used in the screenshot has been attached below.

exploit_folder.zip

2. Run the executable with this line:

```
vsim  ../Project/ModelSim/Main.TestMain  +MAX_TEST_CYCLES=50000
+TEST_CODE=exploit_folder/ +ENABLE_PC_GOAL=1 +SHOW_SERIAL_OUT=1 -
suppress 8233 +autofindloop  +RSD_LOG_FILE=RSD.log
+DEBUG_LOG_FILE=Debug.log  +REG_CSV_FILE=Register.csv
```

Proof-of-Concept Exploit

This PoC exploit demonstrates how the unchecked CSR value can be exploited.

1. In Group 1, the FS flag in the mstatus register is turned off by creating a bit mask and clearing those bits.
2. In Group 2, NOP instructions allow the processor enough time to commit the instructions.
3. In Group 3, an instruction operates on a floating-point register. This should throw an illegal exception because the FS flag is “Off”.
4. In Group 4, NOP instructions allow the processor enough time to commit the instructions.

```
// GROUP 1: set FS in mstatus to 00 Off
asm volatile ( "CSRR t0, mstatus" ); // store mstatus register to t0
asm volatile ( "LI t1, 0xFFFF9FF" ); // create bitmask for FS flag in
t1
asm volatile ( "AND t0, t0, t1" ); // clear FS bits
asm volatile ( "CSRW mstatus, t0" ); // write new FS flag back to
mstatus

// GROUP 2: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 3: write to a floating point register
asm volatile ( "LI a0, 0x3f800000" ); // set value to temporary
register a0
```

```
asm volatile ( "FMV.W.X f1, a0" ); // this instruction should be
illegal

// GROUP 4: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
```

Possible Causes

RSD decoding logic at [`Processor/Src/Decoder/Decoder.sv`] for floating-pointer registers does not account for `mstatus` CSR values. For example, `RISCV_DecodeOp` has no checks on whether the processor has enabled or disabled access to certain floating-point registers.